

Secure Software - Intermediate delivery

Business goal: Provide a web platform that allows users to store and manage documents in digital format.

Assets:

- Physical
 - DB Server
 - Storage Bucket
 - Application Server
- Logical
 - Documents and metadata
 - User credentials
 - Source code

Threat Analysis

Critical Assets: User docs, S3 Bucket, credentials

Trust Boundaries:

- Application Server - PostgreSQL
- Application Server - S3 Bucket
- Application Server - Client

STRIDE Analysis:

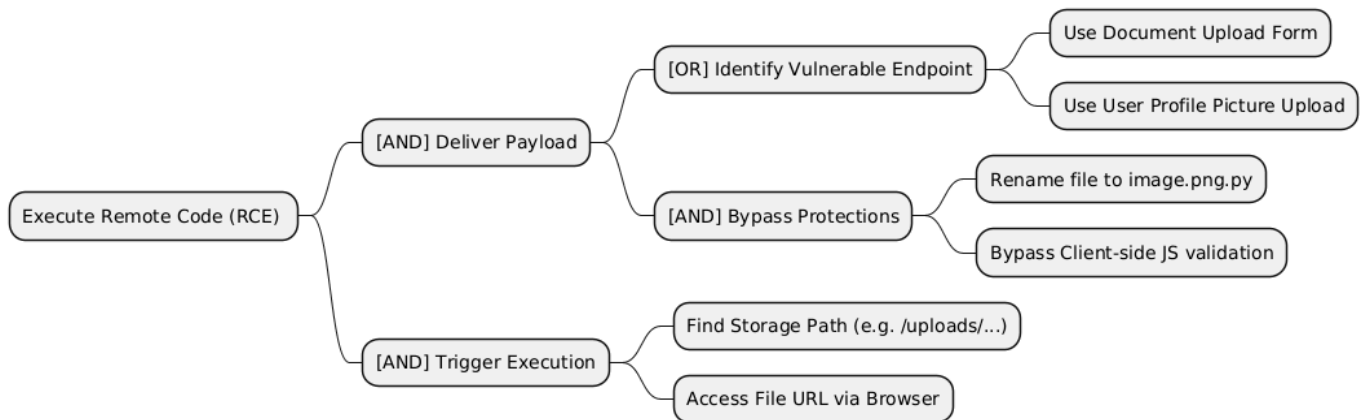
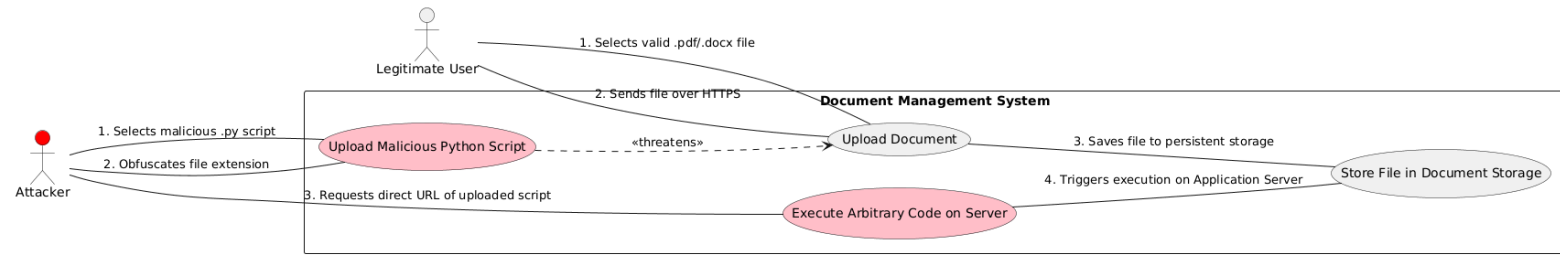
Threat Type	Mitigation Techniques
Spoofing Identity	1. Appropriate authentication 2. Protect secret data 3. Don't store secrets
Tampering with data	1. Appropriate authorization 2. Hashes 3. MACs 4. Digital signatures 5. Tamper resistant protocols
Repudiation	1. Digital signatures 2. Timestamps 3. Audit trails
Information Disclosure	1. Authorization 2. Privacy-enhanced protocols 3. Encryption 4. Protect secrets 5. Don't store secrets
Denial of Service	1. Appropriate authentication 2. Appropriate authorization 3. Filtering 4. Throttling 5. Quality of service
Elevation of privilege	1. Run with least privilege

Misuse Scenarios:

- 1: Remote Code Execution (RCE) via Malicious Upload.** A user uploads a malicious file (e.g., a Python script) instead of a standard document to achieve arbitrary code execution on the application server.
- 2: Unauthorized Document Access by Reviewer.** A Reviewer attempts to download documents that have not been explicitly shared with them, bypassing intended access restrictions.
- 3: Privilege Escalation.** An authenticated user exploits a code vulnerability to access administrative routes, successfully listing all registered users or disabling other users' accounts.
- 4: Brute Force / Credential Stuffing.** An external attacker attempts to guess user passwords via the login page to gain unauthorized access to the platform.
- 5: Session Hijacking.** An attacker intercepts or steals a valid session cookie to impersonate a User, Reviewer, or Administrator without providing valid credentials.
- 6: Insecure Direct Object Reference (IDOR) in Download.** A user attempts to download documents belonging to others by modifying the file ID in the URL or API request, exploiting a lack of server-side ownership verification.
- 7: Persistent Access after Revocation.** A Reviewer continues to access a document after the owner has revoked their permissions, due to flawed authorization state management in the PostgreSQL database.
- 8: Role-Based Access Control (RBAC) Bypass.** A user with the "User" role attempts to send direct requests to "Administrator" endpoints to view the global directory of registered users.
- 9: SQL Injection (SQLi).** An attacker injects malicious SQL commands into the document search parameters to exfiltrate sensitive information (such as password hashes) from the PostgreSQL database.
- 10: Stored Cross-Site Scripting (XSS) via Metadata.** A user names an uploaded document using a JavaScript snippet; the code executes in the browser of an Administrator or Reviewer when they view the document list on the platform.
- 11: Denial of Service (DoS) via Storage Saturation.** A malicious user repeatedly uploads large files to exhaust the storage system's disk space, rendering the application unable to accept new legitimate documents.
- 12: Malicious Dependency Injection (Supply Chain Attack).** Exploiting a lack of security controls in the original CI/CD workflow, an attacker introduces a vulnerable or compromised Python library during the artifact build and packaging phase.
- 13: Accidental Exposure of Secrets.** Database credentials or application secret keys are inadvertently printed within the CI/CD pipeline logs, making them visible to any stakeholder with access to the repository or build history.

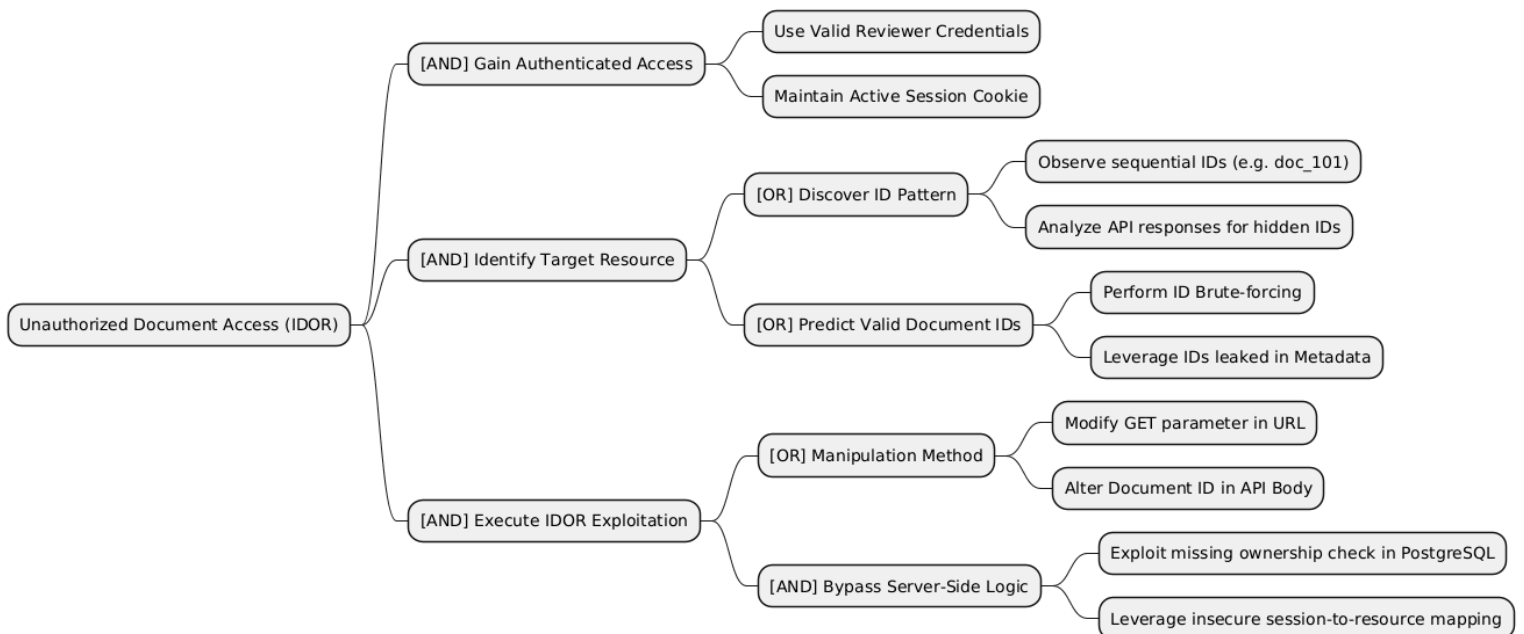
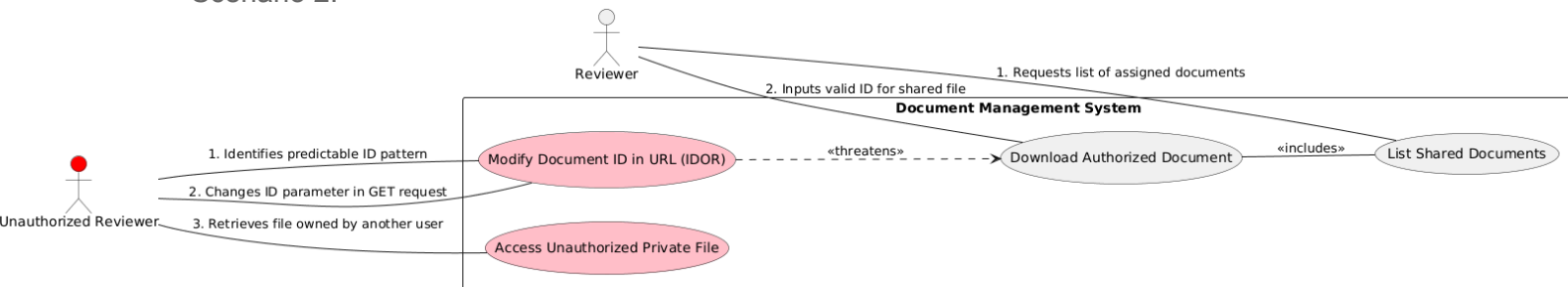
Misuse Case Diagrams and Attack Vectors:

Scenario 1:



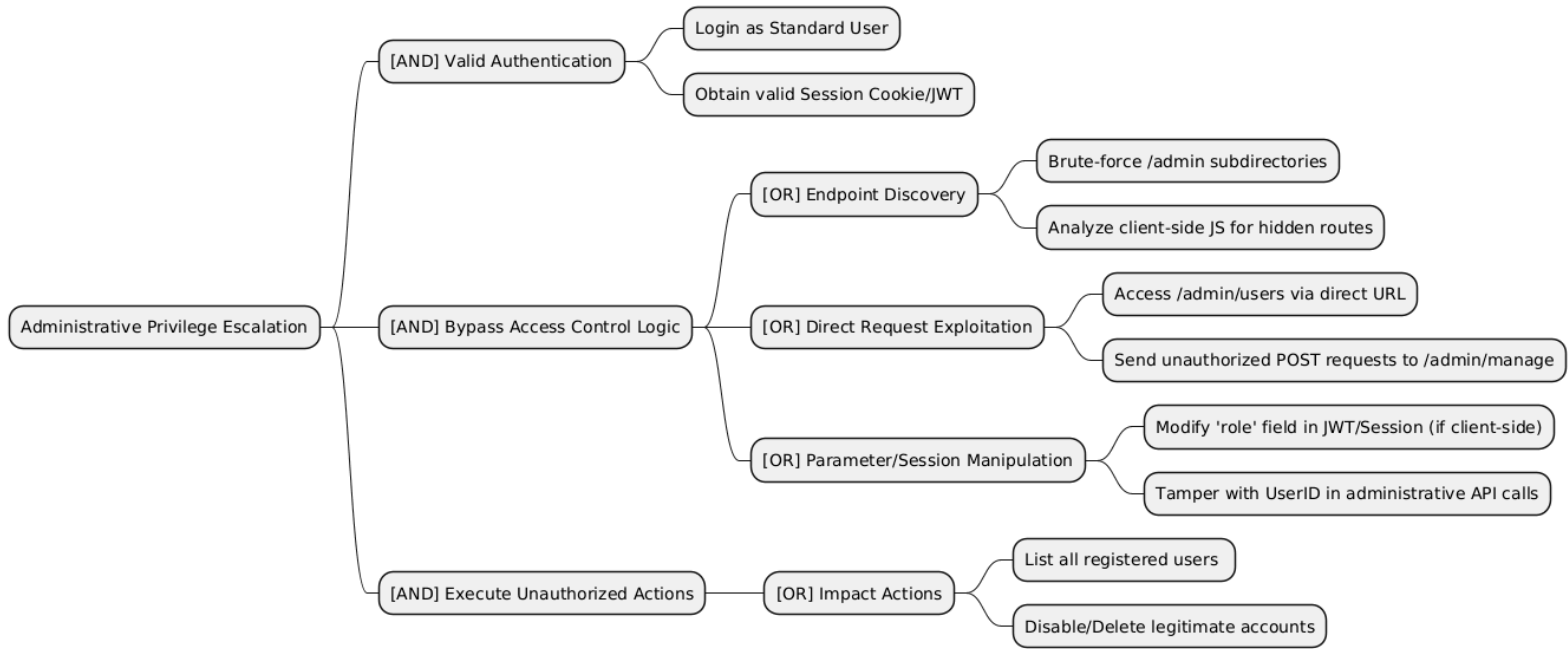
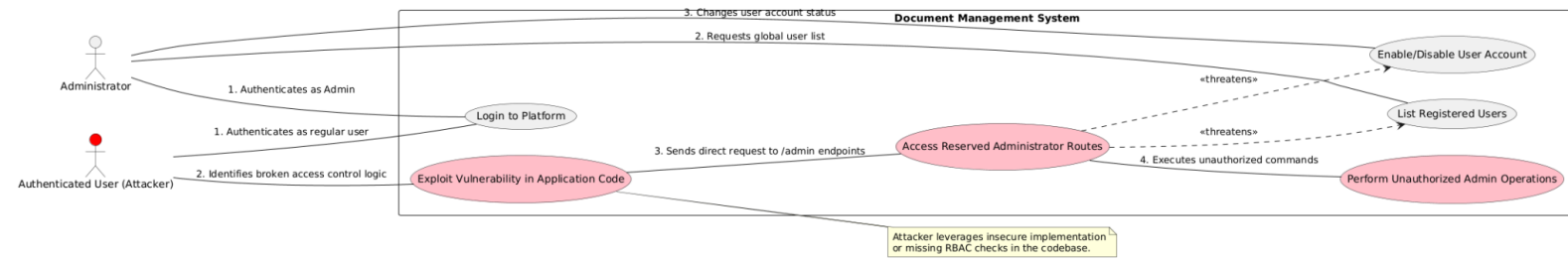
Attack Tree: Remote Code Execution via File Upload

Scenario 2:



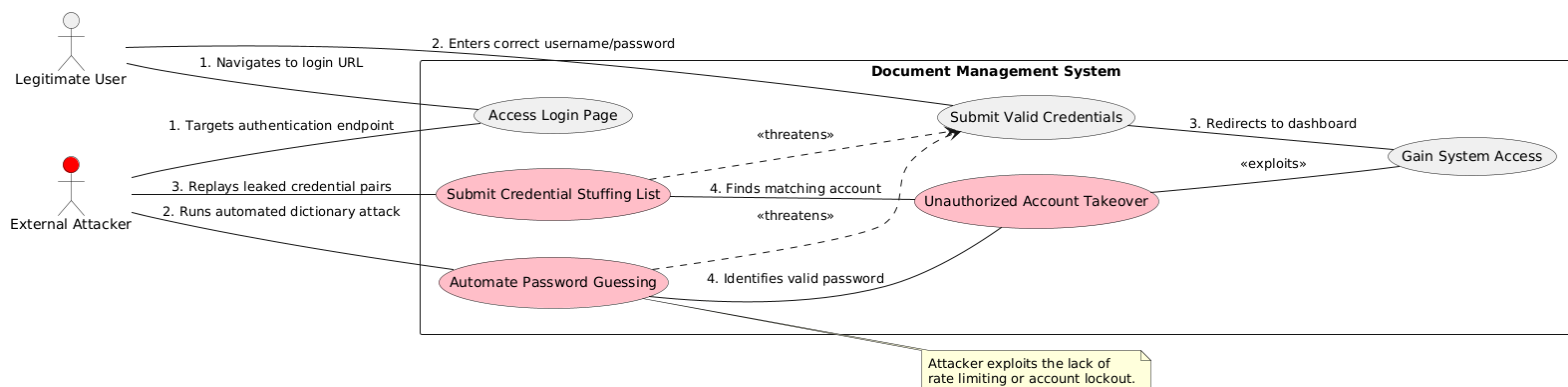
Attack Tree: Unauthorized Access via IDOR (Scenario 2)

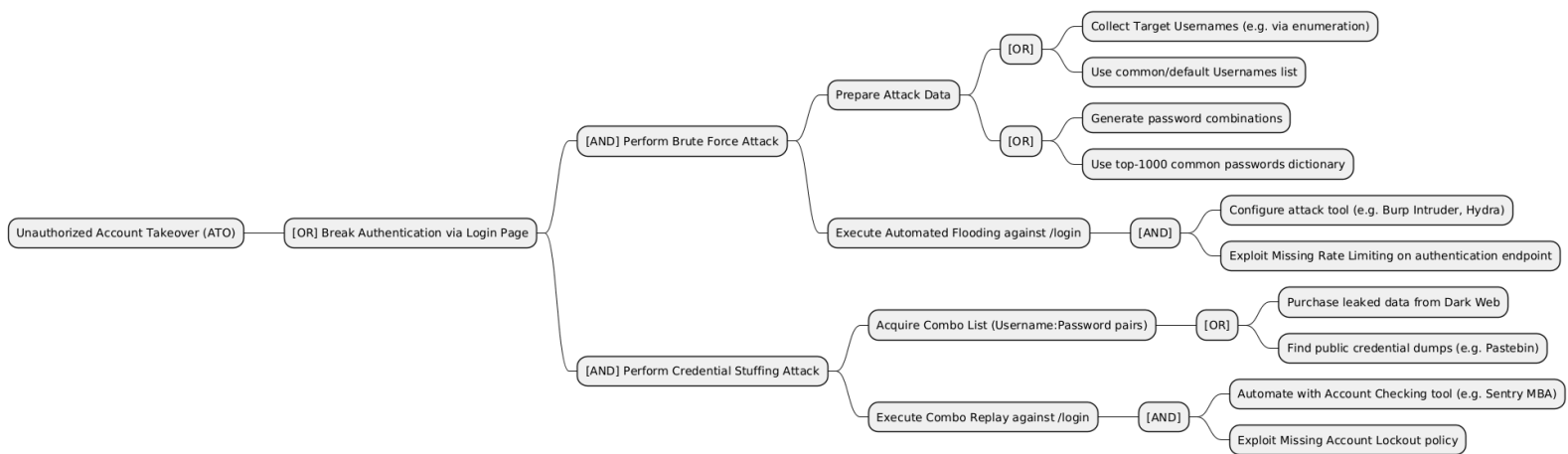
Scenario 3:



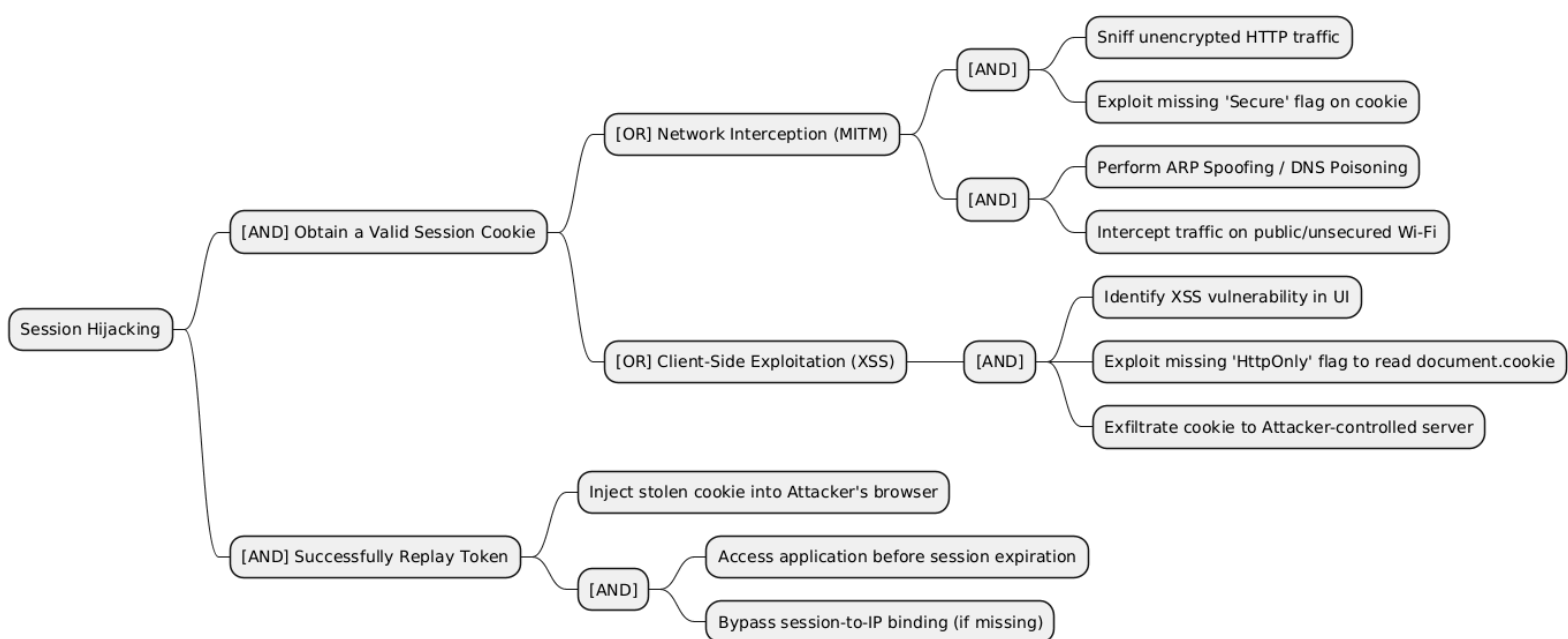
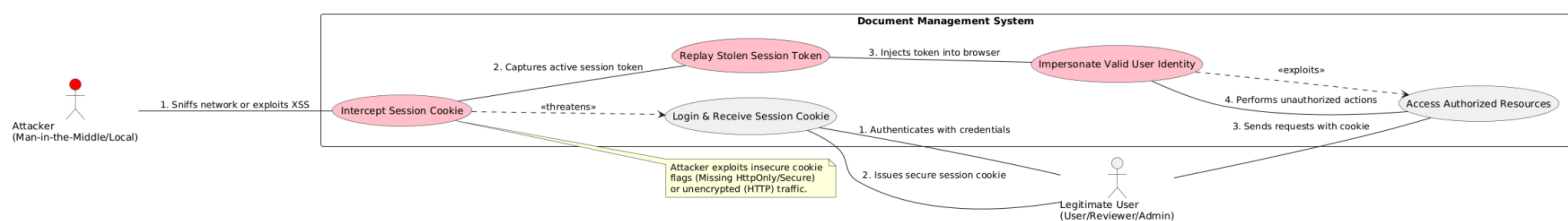
Attack Tree: Privilege Escalation via Code Vulnerability (Scenario 3)

Scenario 4:

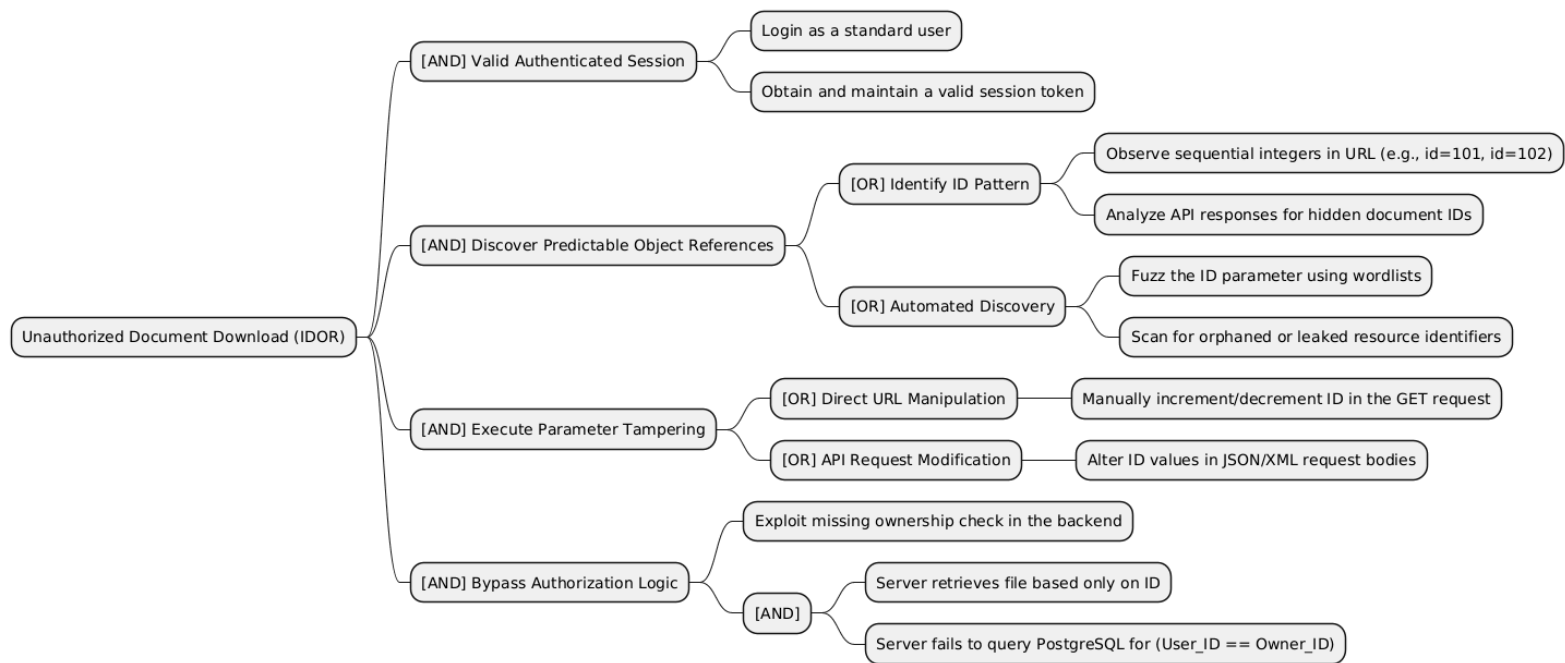
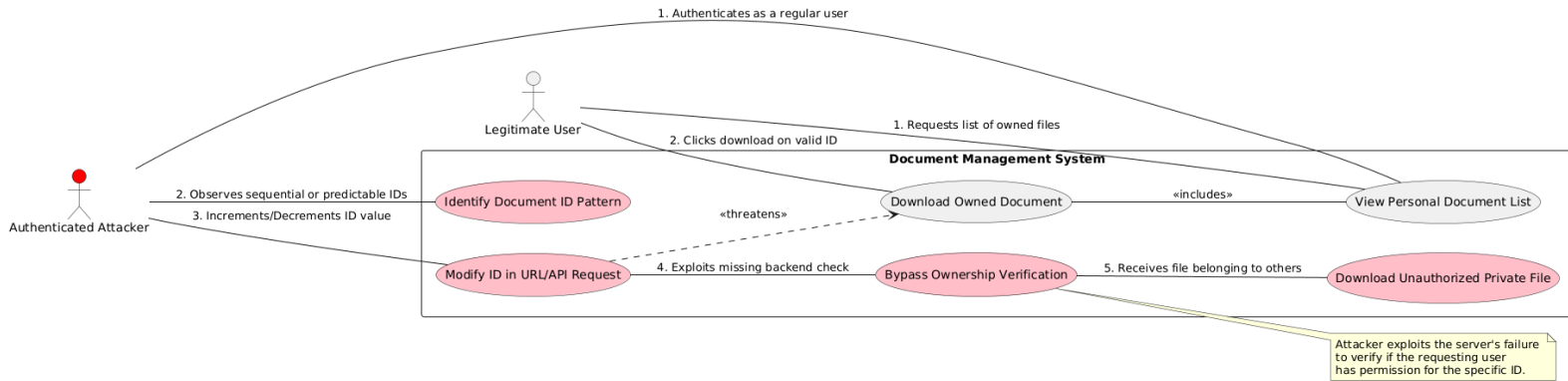




Scenario 5:

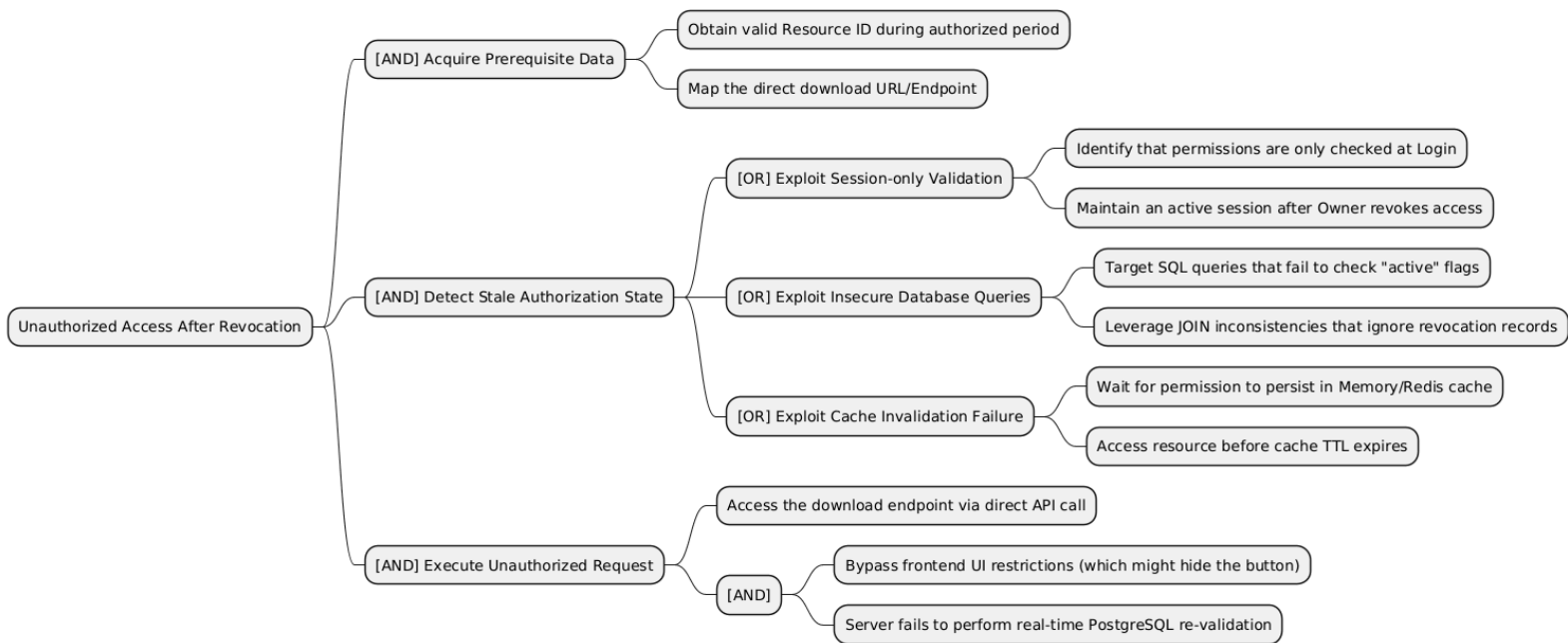
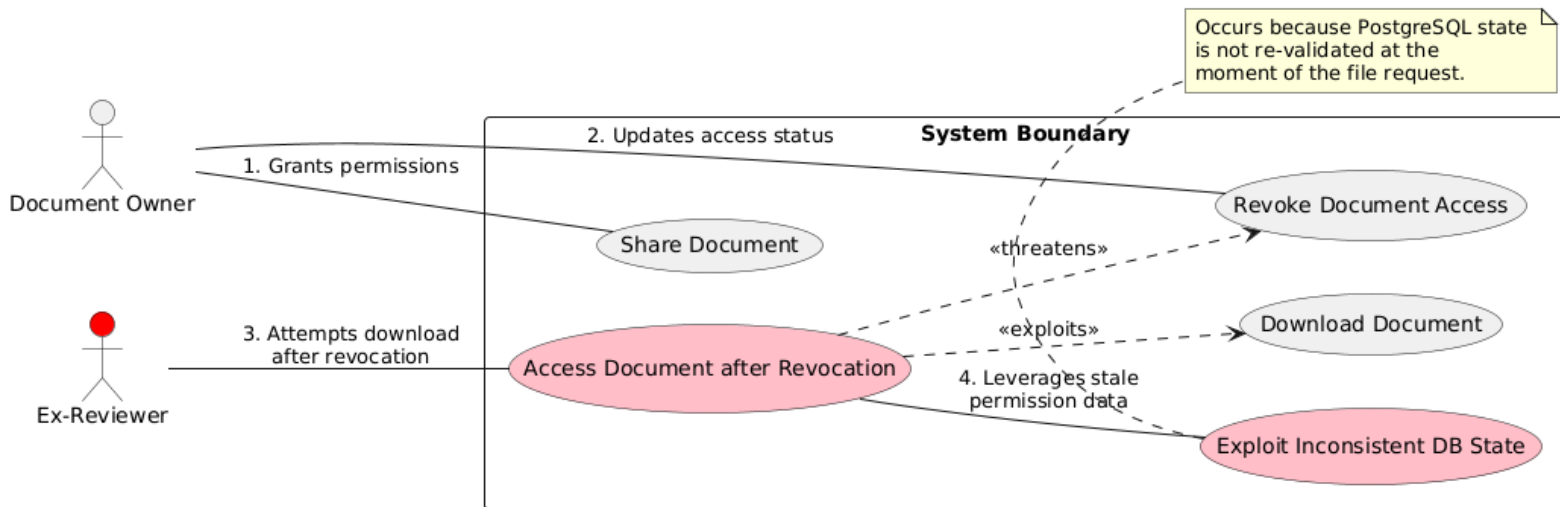


Scenario 6:



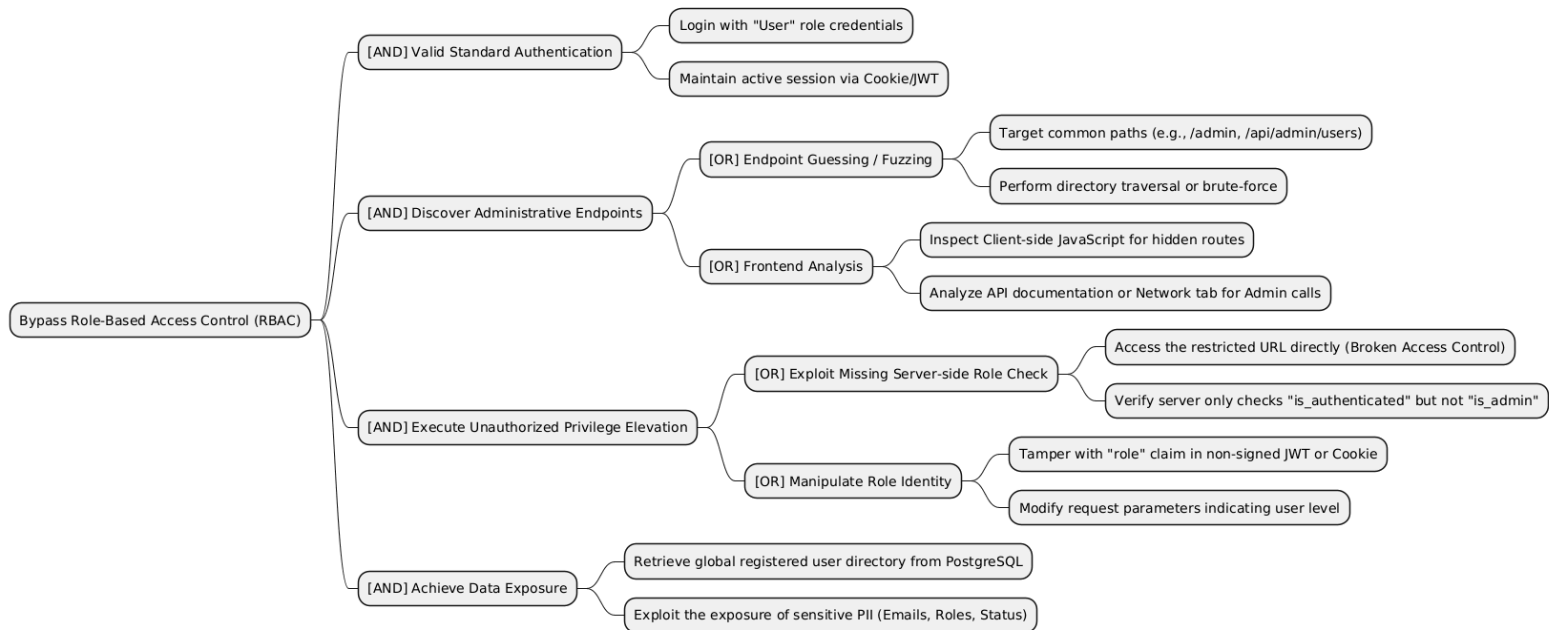
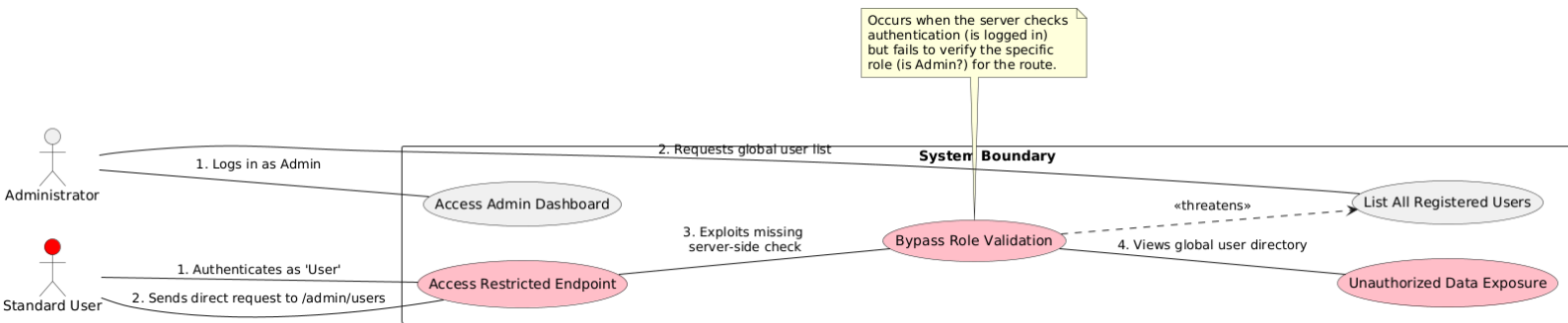
Attack Tree: IDOR Exploitation for Document Download (Scenario 6)

Scenario 7:



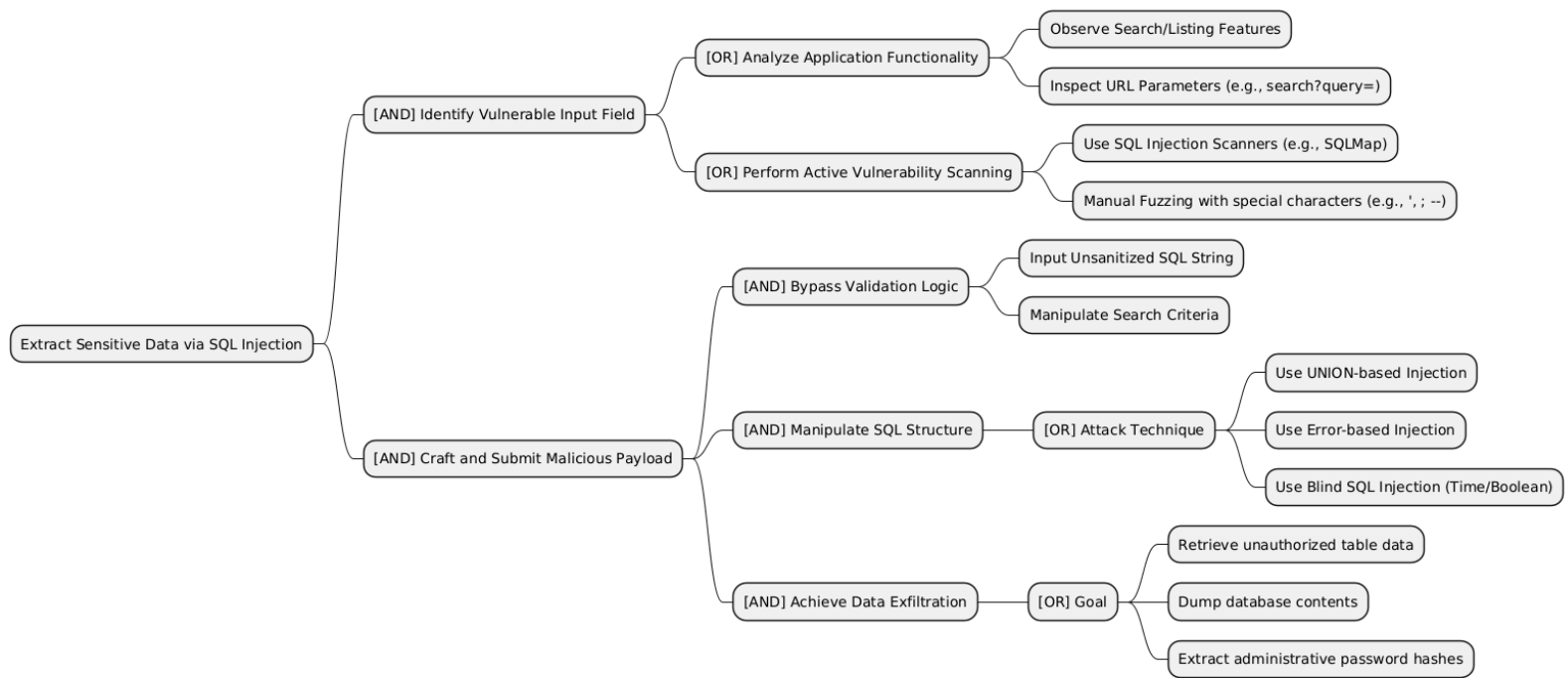
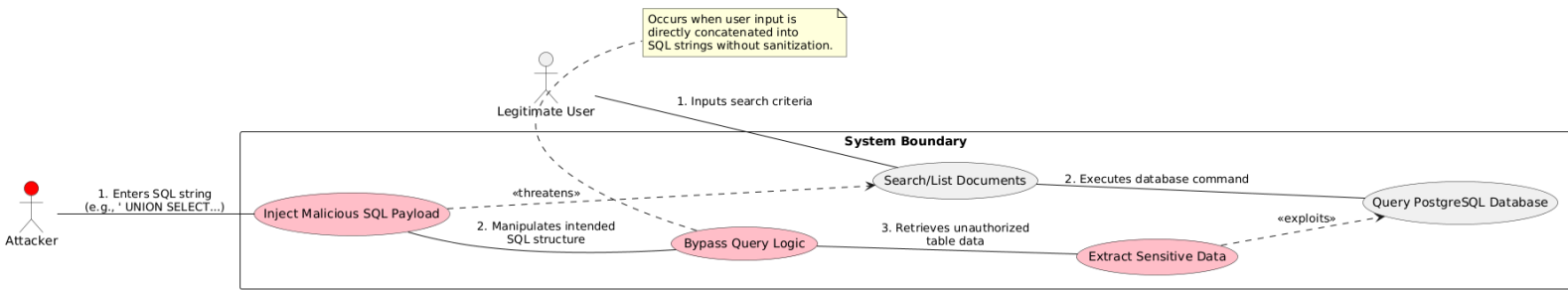
Attack Tree: Persistent Access Post-Revocation (Scenario 7)

Scenario 8:



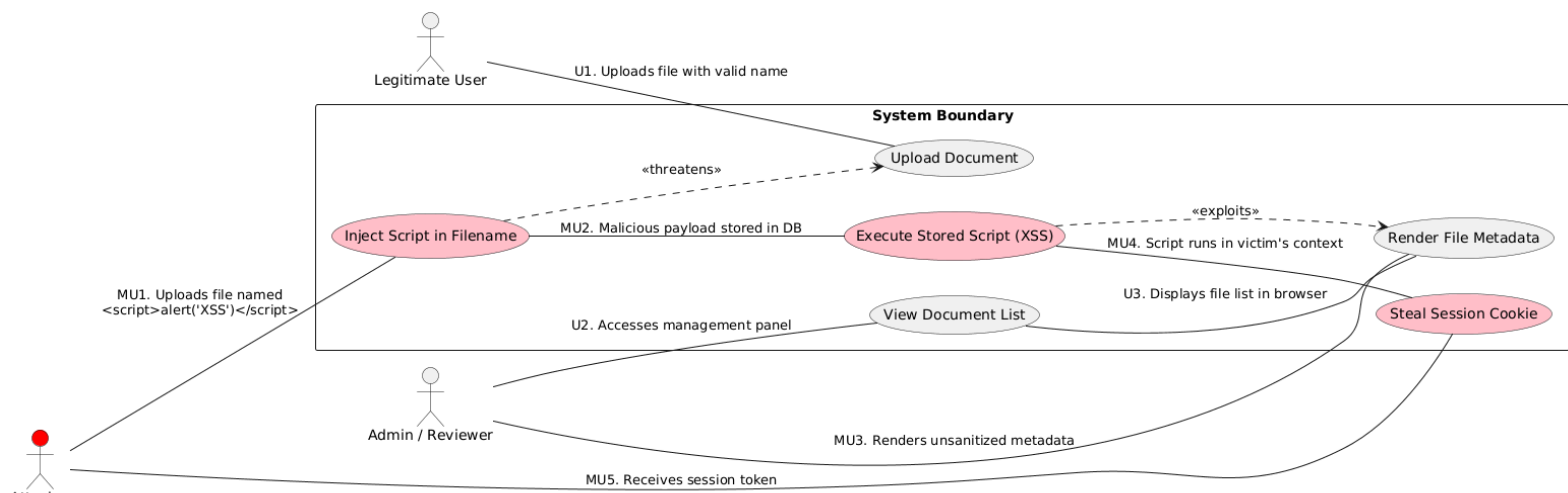
Attack Tree: Administrative RBAC Bypass (Scenario 8)

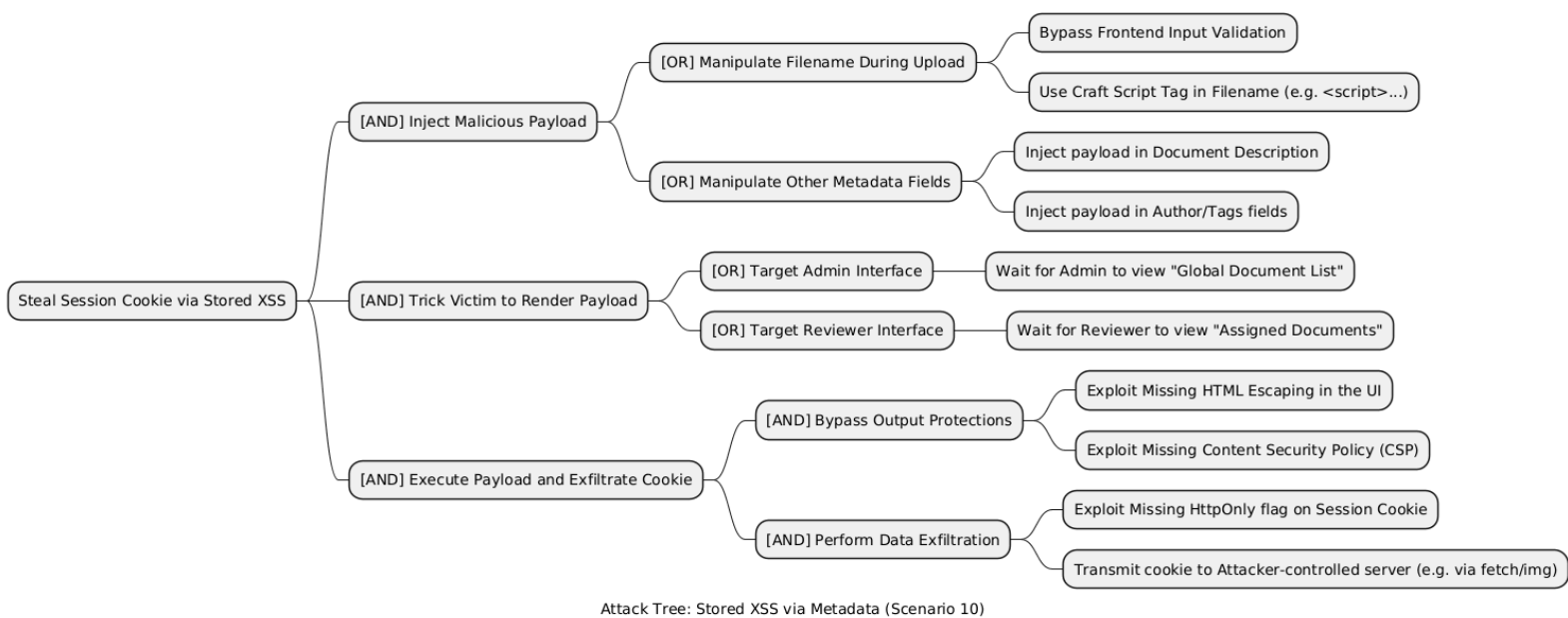
Scenario 9:



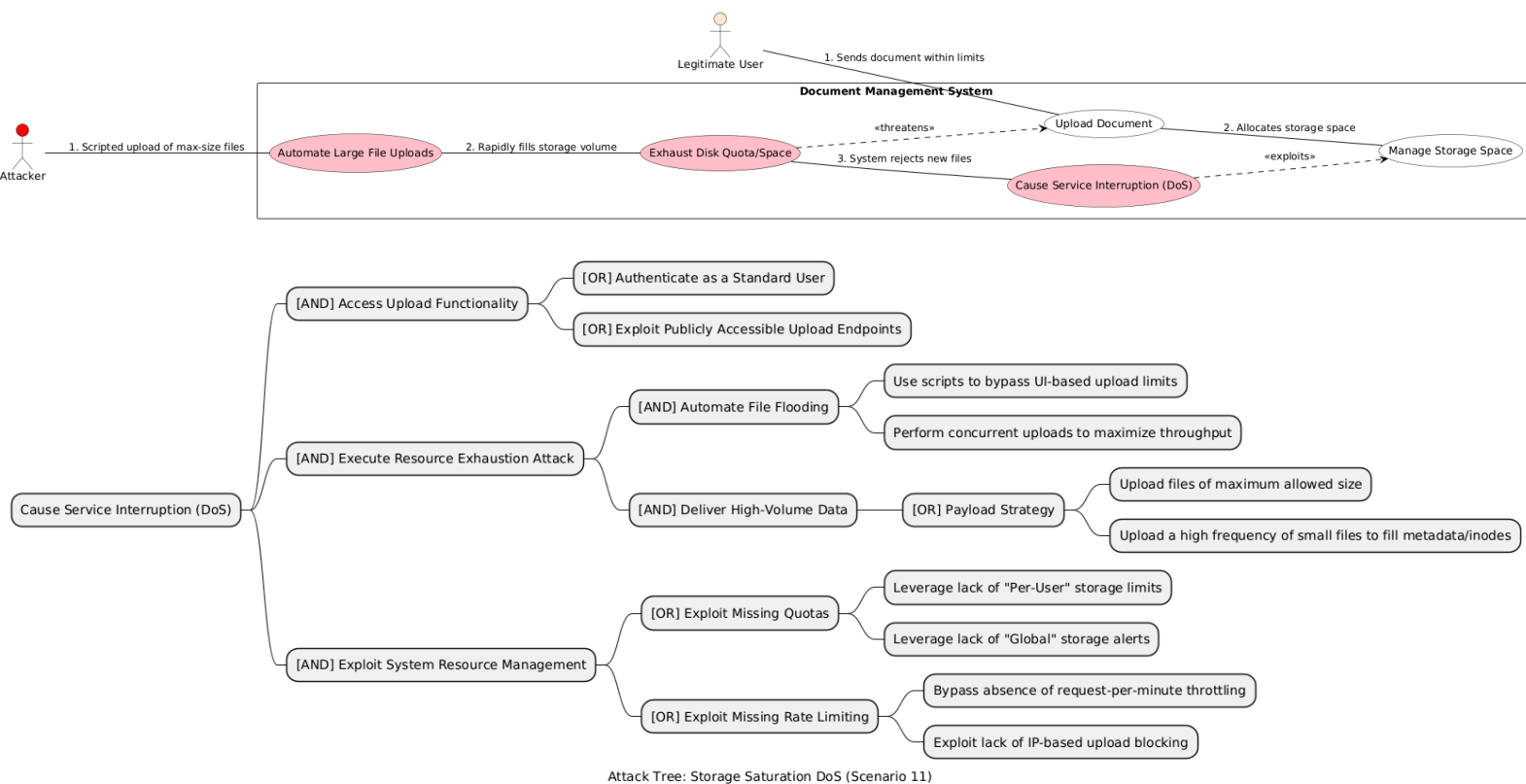
Attack Tree: SQL Injection (SQLi) via Document Search (Scenario 9)

Scenario 10:

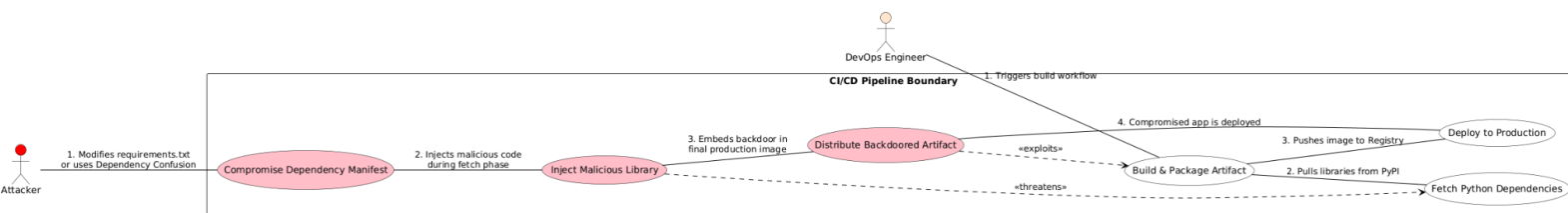


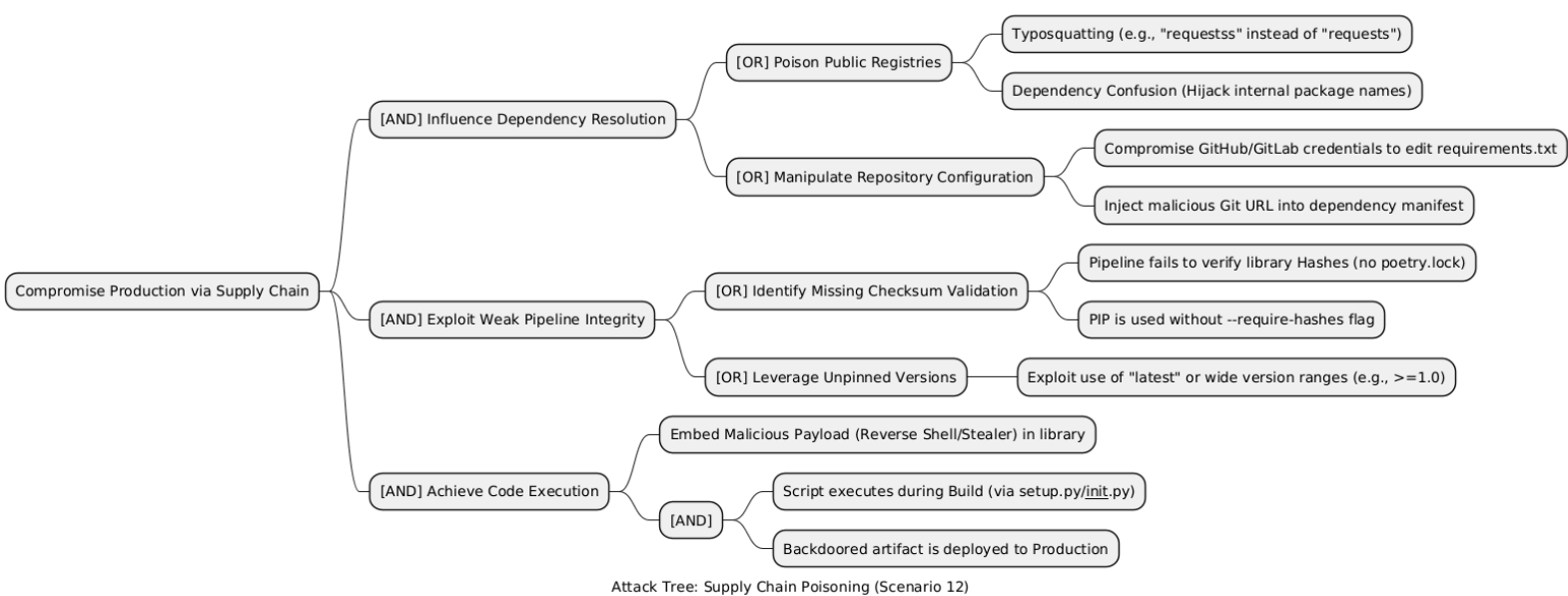


Scenario 11:

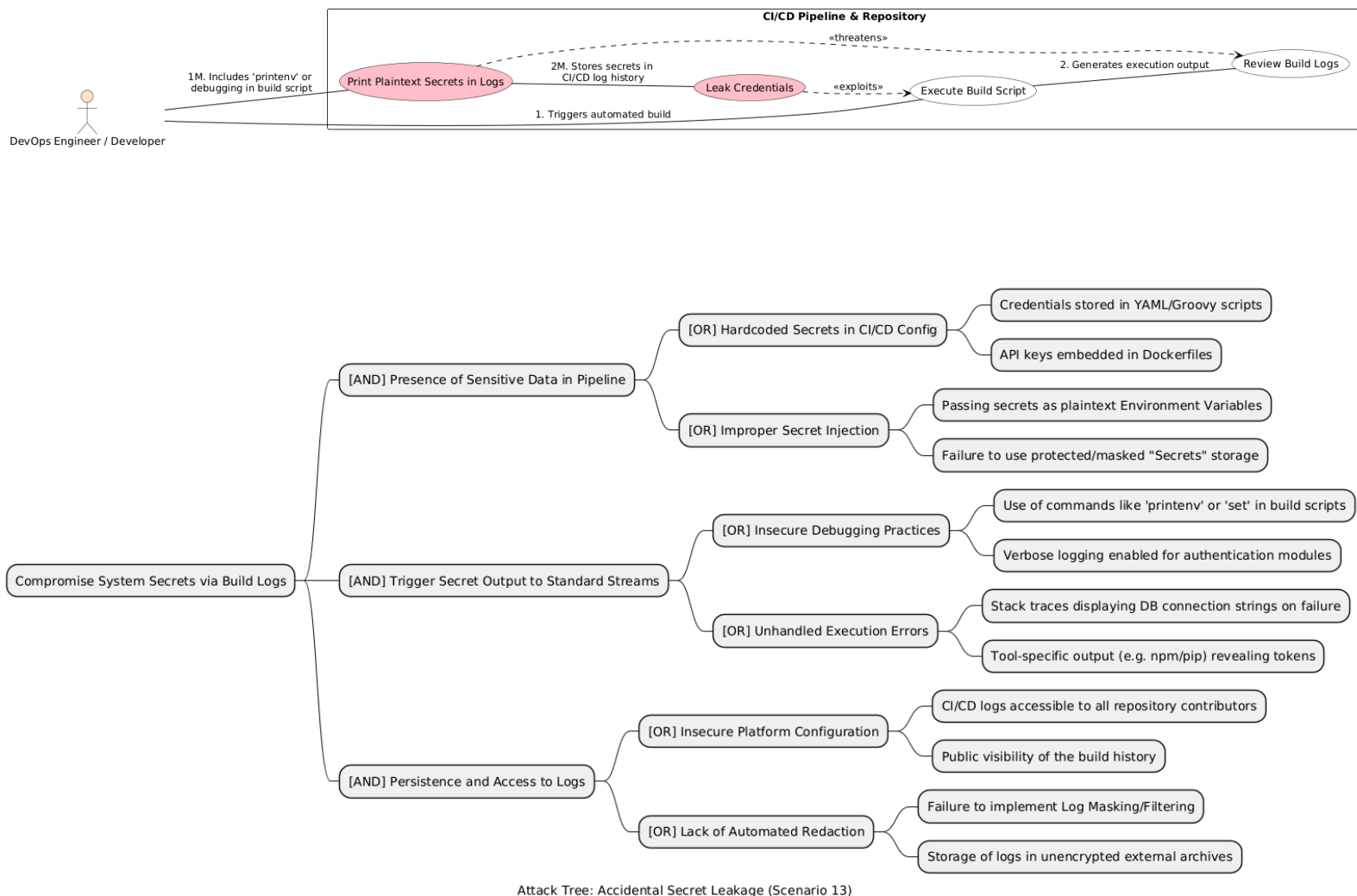


Scenario 12:





Scenario 13:



Security Goals

1. **Document Confidentiality**: Protect uploaded documents, which may contain sensitive or private information, against unauthorized access by third parties.
2. **Role-Based Access Control (RBAC)**: Ensure that users perform only authorized actions, enforcing strict permission boundaries (e.g., restricting a Reviewer's access to shared files only).
3. **Robust Authentication**: Verify the identity of all users (User, Reviewer, Administrator) prior to granting any interaction with the system.
4. **Data and Metadata Integrity**: Guarantee that documents and data stored within the PostgreSQL database remain immutable and protected from unauthorized modification or corruption.
5. **User Session Security**: Prevent session hijacking and unauthorized interception of active user sessions during platform interaction.
6. **Principle of Least Privilege**: Configure system components to operate with the minimum set of permissions strictly necessary for their intended function.
7. **Resource and Trust Boundary Isolation**: Maintain clear architectural separation between the client browser (untrusted), the application server, and the document storage to restrict potential lateral movement by an attacker.
8. **Secure Delivery Pipeline**: Safeguard the integrity of the CI/CD pipeline to prevent the introduction of vulnerable or malicious code during the build and packaging processes.
9. **Accountability for Administrative Actions**: Ensure full auditability of all critical administrative actions, such as user provisioning or privilege management.

10. **Service Availability:** Maintain the accessibility of the platform and its core functionalities (upload, download, listing) in accordance with operational requirements.

Security Goals Classification and Prioritization

Low Priority

Service Availability Reason: In an academic document management system, brief periods of downtime are considered less severe than the potential loss or theft of sensitive user data.

Accountability for Administrative Actions Reason: While essential for oversight, auditing chronologically occurs after access has been granted; preventing abuse is more urgent than logging it after the fact.

Medium Priority

Secure Delivery Pipeline (DevSecOps) Reason: At this intermediate stage, the integrity of the build process is secondary to the immediate protection of data already residing within the system.

Resource and Trust Boundary Isolation Reason: This serves to limit the attack surface. It is an essential "Defense in Depth" principle for a secure architecture, but its effectiveness depends on the correctness of logical controls.

Principle of Least Privilege Reason: This reduces the blast radius of a potential breach. It ensures that if a single component is compromised, the resulting damage remains contained.

User Session Security Reason: As a web-based service, session hijacking would render even the strongest authentication mechanisms redundant.

High Priority

Data and Metadata Integrity Reason: The platform must guarantee that uploaded documents remain unaltered. A sensitive document modified by a malicious actor loses its legal, operational, and evidentiary value.

Robust Authentication Reason: This is the primary point of contact and the foundation of the entire security posture. If user identity is not correctly verified, all subsequent controls become ineffective.

Role-Based Access Control (RBAC) Reason: The system explicitly defines three roles with distinct permissions (User, Reviewer, Admin). Ensuring that a user cannot access another's files is the most critical functional security requirement.

Document Confidentiality Reason: Since documents may contain private or sensitive information, the failure of this goal represents the highest possible impact and damage to stakeholders.

Security Requirements

Security Requirements are technical and functional specifications that define how the system must behave to mitigate identified threats and protect assets.

1. Access Control

- **Asset Ownership:** The system must verify user authorization for every download request; access is granted only if the user is the owner or an authorized Reviewer.
- **Role Separation:** Account management functionalities must be accessible exclusively to users with the Administrator role.
- **Least Privilege:** Users with the Reviewer role must have read-only permissions and shall not be able to rename, delete, or upload files on behalf of third parties.

2. Authentication and Session Management

- **Password Management:** The system must store passwords exclusively using secure, industry-standard hashing algorithms.
- **Session Protection:** Session cookies must be configured with HttpOnly and Secure flags to enforce HTTPS.
- **Brute Force Prevention:** The system must implement a rate-limiting mechanism or a temporary account lockout after 5 consecutive failed login attempts on a single account.

3. Data Integrity and Security

- **Input Sanitization:** All interactions with the PostgreSQL database must utilize parameterized queries to neutralize SQL Injection attacks.

- Output Encoding: All data retrieved from the database and rendered in the browser must be encoded to prevent Stored XSS attacks.

4. Monitoring and Accountability

- Audit Logging: Every administrative action and modification to sharing permissions must generate a log containing: timestamp, actor ID, action type, and result.
- Error Masking: In the event of a server error, the system must display a generic message to the user, preventing the leakage of technical stack information.

5. Pipeline Security

- Vulnerability Scanning: The CI/CD pipeline must perform automated dependency scanning on every commit and block the build if critical vulnerabilities are detected.
- Static Analysis: Python code must be analyzed using SAST (Static Application Security Testing) tools to detect insecure functions prior to deployment.

6. File System Integrity

- Storage Isolation: Uploaded files must be stored in a directory outside the web server's "document root", and file system permissions must explicitly prevent execution within that directory.

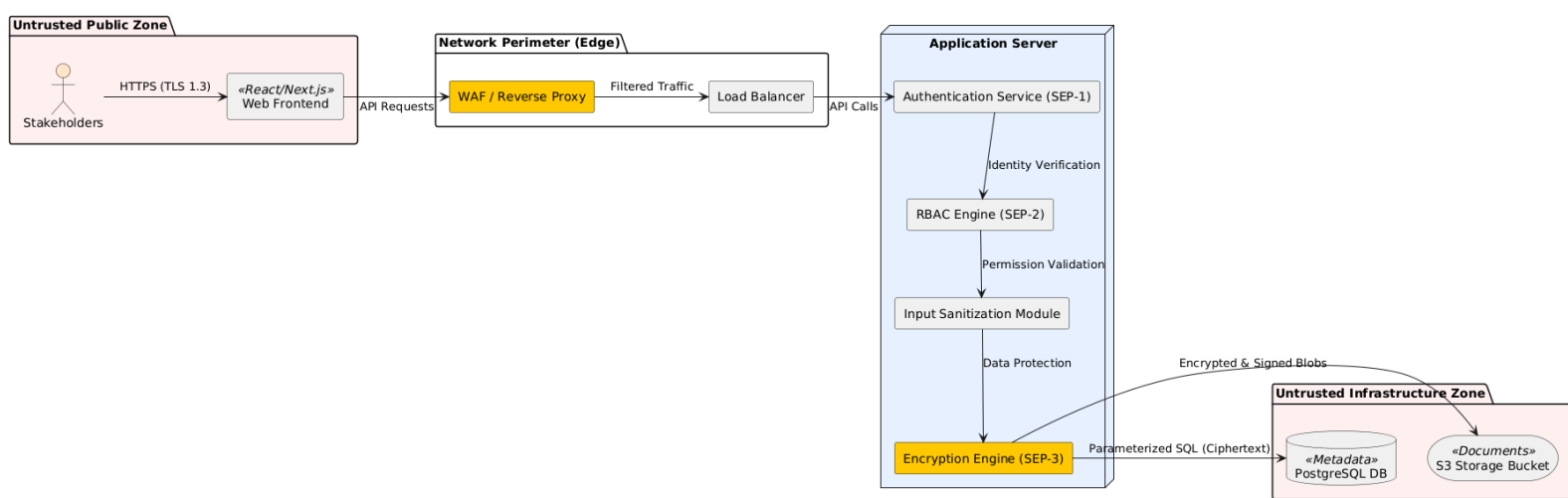
7. Availability and Rate Limiting

- Resource Size Limit: The system must enforce a maximum file size limit for uploads to prevent disk exhaustion attacks.

8. Administrative Traceability

- Account Management Logging: Whenever an Admin enables or disables a user, the system must record the Admin ID, the modified User ID, and the justification for the action.

Proposed Secure Architecture



The system architecture has been designed following the Defense in Depth principle, structuring the application into independent logical layers. This approach ensures that the compromise of a single module does not automatically lead to the security collapse of the entire system.

2.1 Layered Architecture

The system design adopts a Defense in Depth strategy, segmenting the architecture into five distinct layers to isolate critical assets and strictly enforce trust boundaries

- **Network Perimeter:** Managed by a WAF and Load Balancer to handle TLS 1.3 termination and rate-limiting. This layer mitigates brute-force attempts and storage-based DoS attacks
- **Presentation Layer:** A React-based frontend that implements security headers and output encoding. It is designed to neutralize Stored XSS vulnerabilities by sanitizing data before browser rendering
- **Application Layer (Trusted Enclave):** The centralized decision-making core that manages robust authentication (SEP-1) and the RBAC engine (SEP-2) . It enforces mandatory asset ownership verification for every request
- **Persistence Layer (Untrusted):** Comprises the PostgreSQL database and S3 storage, which are logically isolated from the public network . This layer accepts only salted hashes or encrypted ciphertext (AES-256), utilizing parameterized queries to eliminate SQL Injection risks
- **Delivery Layer (DevSecOps):** A secure CI/CD pipeline that integrates automated SAST (Static Application Security Testing) and dependency scanning . This ensures the integrity of the build process and prevents supply chain compromises

2.2 Identification of Trust Boundaries

Trust Boundaries define the logical borders where data is transferred between zones with different privilege levels. Their correct identification is essential for the proper placement of security controls:

- **User-Application Boundary:** This boundary separates the user environment (browser) from the Application Server. It is the critical entry point where the application assumes all input is potentially malicious. Authentication, session management, and input validation mechanisms reside here.
- **Application-Database Boundary:** This boundary separates the business logic from the PostgreSQL database. The architecture requires the Application Server to communicate with the database exclusively through parameterized queries, isolating the system state from SQL injection attempts.
- **Application-Storage Boundary:** This boundary separates the business process from the File System where documents are stored. Access to files is never direct via URL but is mediated by an authorization component that verifies permissions before retrieving content.

2.3 Security Mechanisms

To monitor the identified boundaries and ensure compliance with security requirements, the following defense mechanisms have been integrated:

- **Policy Enforcement Point (PEP) & Authorization Middleware:** Implemented as a central component of the Application Server, the PEP intercepts every request directed at protected resources. Before executing any read or modify operation, the middleware queries the access control system to verify that the authenticated user possesses the necessary rights for the specific resource.
- **Input Sanitization & Output Encoding:** To prevent Injection and XSS vulnerabilities, all incoming data is validated against a predefined schema (types, lengths, formats), while data retrieved from the database is encoded before being rendered in the browser.
- **Secure Storage Hardening:** Files uploaded by users are not stored with their original names; instead, they are renamed using unique identifiers and stored in a directory that is not publicly accessible by the web server. File system access is permitted exclusively to the Application Server, following strict permission validation.
- **Session Management & Hardening:** The session lifecycle is protected by setting cookies with HttpOnly and Secure flags. This mechanism prevents token theft via client-side scripts and forces transmission exclusively over encrypted channels, ensuring session integrity during transit.
- **Audit Logging & Error Masking:** Every critical operation (e.g., privileged access, permission modification) is recorded in immutable logs. Simultaneously, the system implements an error-handling interface that masks technical exceptions, replacing them with generic messages for the end user to prevent Information Leakage.